



Agents for Genome-scale Metabolic Modeling: Automated Construction, Simulation and Analysis

A Thesis Submitted in Partial Fulfilment of the Requirements for the
Interdisciplinary Dual Degree in Biological Engineering and Data Science

by

Saathvik Sista

Roll Number: BE21B031

Under the guidance of

Dr. Meiyappan Lakshmanan

Department of Biotechnology

IBSE, WSAI

IIT Madras

Dr. Nirav Pravinbhai Bhatt

Department of Biotechnology

IBSE, WSAI

IIT Madras

June 2026

Acknowledgements

I am deeply grateful to my supervisors, Prof. Nirav Bhatt and Prof. Meiyappan Lakshmanan, for their guidance, patience, and sustained mentorship throughout this project; this work would not have been possible without them. I would also like to thank Pavan Kumar for the many discussions that shaped the direction of the analysis. My sincere thanks to Adhil Ahmed, who was available to help with technical aspects of the BiSECT lab's compute infrastructure made the experimental work possible. I am grateful to the BiSECT lab and its members for the collaborative environment and shared resources that this project relied on. Finally, I thank the Department of Biotechnology and the Department of Data Science and Artificial Intelligence at IIT Madras for their institutional support, and IBSE for providing the compute resources that enabled this work.

Abstract

Genome-scale metabolic (GSM) modelling is a powerful constraint-based computational framework for the systematic analysis of cellular metabolism under genetic and environmental perturbations. The biological insights these analyses yield hold direct and actionable value for domain-focused experimental scientists whose expertise lies in the biology itself. Despite its broad applicability, however, GSM analysis imposes a high technical entry barrier: conducting even a standard study demands programming proficiency, familiarity with rapidly evolving software ecosystems, and the ability to coordinate complex, multi-step analytical workflows. Existing platforms partially address usability but remain fragmented — each covers only a narrow segment of the modelling lifecycle, offers rigid template-driven pipelines with little flexibility, and forces researchers to switch between multiple tools to complete a single study, placing the full power of GSM analysis out of reach for the majority of researchers.

This thesis presents **MetaPilot**, an AI-enabled web application that allows domain-focused researchers — including experimental biologists without systems biology programming expertise — to design and execute comprehensive, multi-step GSM workflows using natural language. MetaPilot operates in two modes: **MetaInteract**, an agentic chat interface where natural language instructions drive the interrogation, modification, and analysis of genome-scale metabolic models through a suite of almost **40** COBRApy-backed tools spanning the full modelling lifecycle — from model reconstruction and curation through constraint definition, optimisation, sensitivity analysis, and visualisation — and **MetaPlan**, where the system accepts a high-level research objective and returns a structured, user-editable, multi-step analysis plan that is executed and interpreted by the agent end-to-end. The multi-step planning capability of MetaPlan is grounded in a Procedural Knowledge Base (PKB) construction pipeline developed as part of this work. From an initial corpus of approximately 14,400 GSM research papers, 3,878 curated papers are processed to extract and structure workflow knowledge into a searchable vector knowledge base, that is queried by MetaPlan to synthesize end-to-end analysis plans.

MetaPilot is evaluated through a series of case studies demonstrating end-to-end GSM analysis capabilities, spanning standard single-step analyses through to complex multi-step workflows, including the context-specific reconstruction and metabolic analysis of a human breast cancer cell line under pharmacological perturbation. A qualitative comparison against existing GSM tools further establishes that MetaPilot is the first to offer unified, natural-language- driven access to the full GSM modelling lifecycle without a single line of code, thus democratizing genome-scale metabolic modelling for a broad community of domain-focused researchers.

Keywords: Genome-scale metabolic modelling; Agentic AI; ReAct framework; Retrieval-augmented generation; Procedural Knowledge Base; COBRApy; Flux balance analysis; MetaPilot; Natural language interface; LLM tool-use.

 github.com/NiravBhattLab/Agenti_AI_for_COBRA/tree/MetaPilot

 huggingface.co/datasets/sistasathvik/gsm_procedural_knowledge_base

Contents

Acknowledgements	i
Abstract	ii
1 Introduction	1
1.1 Motivation	1
1.2 Problem Statement	2
1.2.1 Research Objectives	2
1.3 Contributions and Novelty	3
2 Literature Review	3
2.1 Genome-Scale Metabolic Modelling and the COBRA Framework	3
2.2 Existing Tools for GSM Analysis	4
2.3 Agentic AI and the ReAct Framework	4
2.4 Retrieval-Augmented Generation	5
2.5 Research Gap	5
3 Methods	6
3.1 System Architecture	6
3.2 COBRAPy Tool Suite	7
3.3 <i>MetaInteract</i> : Interacting with GSM Models Using Natural Language	8
3.4 <i>MetaPlan</i> : Multi-Step GSM Workflow Planning and Execution	9
3.5 PKB Construction Pipeline	10
3.5.1 Corpus Collection	10
3.5.2 Corpus Filtering	11
3.5.3 Methods Extraction and Step Decomposition	11
3.5.4 Objective Hierarchy Generation	13
3.5.5 Vector Index Construction	15
4 Results	16
4.1 A Comparison of <i>MetaPilot</i> with Existing Tools	16
4.2 Case Studies	17
4.2.1 <i>MetaInteract</i> : Agentic Chat Mode	17
4.2.2 <i>MetaPlan</i> : Multi-step plan mode	19
4.3 Retrieval Quality	20
4.4 PKB Pipeline	21
5 Future Work	23
6 Discussion	24
7 Limitations	24

8 Data and Code Availability	25
List of Abbreviations	26

Section 1: Introduction

Genome-scale metabolic (GSM) modeling is a powerful constraint-based computational framework for systematically analysing cellular metabolism under genetic and environmental perturbations. By encoding the complete set of known biochemical reactions in an organism alongside physicochemical and biological constraints, genome-scale metabolic models (GEMs) provide a structured formalism for investigating how cells distribute metabolic fluxes across their reaction networks in response to changes in nutrient availability, gene expression, or environmental conditions.

Over the past two decades, GEMs have underpinned a diverse range of high-impact biological studies. These include patient-specific analyses of cancer metabolism and the identification of therapeutic targets [1, 2], system-level investigations of host–microbiome metabolic interactions [3, 4], and model-guided optimisation of microbial strains for industrial bioproduction [5]. The ability of GEMs to integrate heterogeneous biological data within a mathematically tractable framework makes them uniquely suited to bridging the gap between molecular knowledge and systems-level phenotypic prediction. The biological insights yielded by these analyses — from identifying metabolic vulnerabilities in disease models to guiding the rational design of production strains — hold direct and actionable value for experimental scientists whose expertise lies in the biology itself; yet realising this value demands a level of computational fluency that remains well outside the reach of most domain-focused researchers.

Recent advances in artificial intelligence — and large language models in particular — have significantly expanded the scope of what is computationally tractable in scientific research, shifting the frontier from predictive modelling and pattern recognition to structured reasoning over complex, domain-specific workflows. Modern LLMs demonstrate the capacity to understand specialised scientific language, follow multi-step analytical reasoning, and interact with external computational tools through function-calling mechanisms. The emergence of agentic LLM systems, which couple this reasoning capacity with autonomous tool invocation and iterative self-correction, enables models to orchestrate end-to-end scientific workflows rather than merely respond to isolated queries. It is this confluence of advances that motivates and underpins the work presented in this thesis.

1.1 Motivation

Despite the breadth and impact of GEM-based studies, designing and executing such analyses remains technically demanding. Researchers are required to possess substantial programming expertise, familiarity with rapidly evolving software ecosystems, and the ability to coordinate complex, multi-step analytical workflows spanning model loading, constraint specification, simulation, and result interpretation. This technical overhead constitutes a significant barrier to adoption, particularly for domain-focused experimental biologists who possess deep biological intuition but limited computational proficiency.

Several tools have been developed to lower this barrier. Web-based platforms such as Fluxer [6] and desktop applications such as OptFlux [7] and CNApy [8] offer graphical interfaces targeting users who require access to common simulation outputs without writing code. COBRAxy [9] integrates constraint-based modelling into the Galaxy platform, enabling the computation of metabolic flux distributions through a browser-based interface. However, these platforms share several fundamental

limitations. Each services only a narrow segment of the modelling lifecycle — covering individual analyses such as FBA or flux visualisation in isolation — rather than supporting end-to-end workflows from model loading through simulation, analysis, and interpretation. Beyond limited coverage, they largely offer rigid, template-driven analysis pipelines that afford little flexibility for exploratory or hypothesis-driven studies. Moreover, the absence of a unified platform means researchers must routinely switch between multiple tools to complete a single study, resulting in fragmented workflows that compound the technical burden rather than alleviating it.

This opportunity has already been seized across adjacent computational biology domains. In single-cell transcriptomics, scChat [10] has demonstrated the transformative potential of LLM-driven assistants: by combining function-calling for quantitative analysis, retrieval-augmented generation for hallucination reduction, and LLM-powered literature integration, it enables context-aware, hypothesis-driven exploration of scRNA-seq data in a manner that was previously inaccessible without significant programming expertise. Similar advances have been made in proteomics with ProtChat [11] and ProtAgents [12], in cheminformatics with ChemCrow [13], and in single-cell spatial transcriptomics with CellAgent [14]. More broadly, tools such as BioOmni [15], AutoBA [16], and the Bioinformatics Agent (BIA) [17] have demonstrated that natural language interfaces can substantially lower the barrier to complex biological analyses. The GSM modelling domain, however, has yet to benefit from equivalent developments, remaining reliant on code-centric, fragmented workflows.

1.2 Problem Statement

The central problem addressed in this thesis is the absence of a unified, natural-language-driven interface for end-to-end GSM analysis. Designing and executing a complete GSM study currently requires a researcher to possess programming expertise, familiarity with the COBRAPy ecosystem, and the ability to manually coordinate multi-step workflows spanning model loading, constraint specification, simulation, result interpretation, and visualisation. This technical overhead excludes a large class of domain-focused researchers who possess the biological knowledge to formulate meaningful research questions but lack the computational proficiency to execute them independently.

A second, deeper challenge compounds this: even if a natural-language interface were provided, a general-purpose LLM has no reliable basis for generating workflows that reflect the methodological conventions of real GSM practice. Without access to structured knowledge of how published GSM studies are actually conducted, a system risks producing plans that are syntactically plausible but methodologically unsound. Grounding multi-step planning in procedural knowledge extracted from published GSM literature is therefore a necessary condition for the system to be scientifically credible.

1.2.1 Research Objectives

The following research objectives are addressed in this thesis:

RO1 To lower the barrier to GSM analysis by enabling researchers to design and execute complex, multi-step analytical workflows through natural language, without requiring programming expertise or familiarity with the underlying computational tooling, and with flexibility in the choice of LLM powering the system.

RO2 To ensure that the system’s multi-step workflow planning capability is grounded in the methodological conventions of real, published GSM practice, rather than relying solely on the general reasoning capabilities of the underlying language model.

1.3 Contributions and Novelty

Prior GSM tools have addressed usability in isolation — offering graphical interfaces for specific analyses or integrating individual methods into browser-based platforms — but none has delivered a unified, natural-language-driven interface spanning the full modelling lifecycle. Agentic AI systems have transformed analytical access in adjacent computational biology domains, yet the GSM field has remained untouched by equivalent developments. Furthermore, no existing system grounds its multi-step planning in structured methodological knowledge drawn from published GSM literature. To our knowledge, this thesis is the first work to:

- **Develop *MetaPilot*, a fully operational, natural-language-driven web application for end-to-end GSM analysis**, supporting both API-hosted and locally deployed LLMs across a Streamlit and FastAPI stack.
- **Design and deploy a dual-mode agentic system for GSM analysis**, in which *MetaInteract* provides a ReAct-based agentic chat interface for natural-language interaction with genome-scale metabolic models across a suite of COBRApy-backed tools spanning the full modelling lifecycle, and *MetaPlan* extends this with a dedicated RAG-based planning agent that retrieves relevant workflows from the Procedural Knowledge Base (PKB), synthesises a structured, user-editable plan, and hands it to the execution agent for end-to-end completion.
- **Construct a Procedural Knowledge Base (PKB) from GSM literature**, processing approximately 3,800 curated papers into a structured ChromaDB vector store that grounds *MetaPlan*’s plan generation in real, published GSM practice.

To the best of our knowledge, this thesis presents the first natural-language-driven agentic system for end-to-end GSM analysis, and the first application of the ReAct framework to the GSM domain. The PKB construction pipeline represents the first automated approach to extracting and structuring procedural knowledge from GSM literature at scale, and the use of this knowledge base to ground multi-step GSM workflow planning via RAG is, to our knowledge, without precedent in the field.

Section 2: Literature Review

2.1 Genome-Scale Metabolic Modelling and the COBRA Framework

Genome-scale metabolic models encode the complete set of biochemical reactions in an organism as a stoichiometric matrix, enabling systematic in silico analysis of cellular metabolism under genetic and environmental perturbations [18]. The foundational analytical technique, Flux Balance Analysis, optimises a biologically meaningful objective — typically biomass production — over a flux space constrained by steady-state mass balance and reaction capacity bounds. Over time, the constraint-based toolkit has expanded considerably beyond FBA to encompass Flux Variability

Analysis, dynamic FBA, geometric FBA, flux sampling, gene essentiality screening, and omics data integration, among others. This proliferation of methods, while scientifically powerful, has correspondingly increased the technical overhead required to design and execute a complete GSM study.

The de facto standard for constraint-based modelling in Python is COBRApy [19], an object-oriented library providing programmatic access to the full spectrum of COBRA methods, with solver abstraction across GLPK, Gurobi, and CPLEX backends and native support for the SBML model format. The tool suite developed in this thesis is built entirely on COBRApy; it is worth noting, however, that the original COBRA Toolbox for MATLAB [20] remains more feature-complete in certain advanced analyses, representing a boundary on the scope of workflows the system currently supports.

2.2 Existing Tools for GSM Analysis

Several tools have been developed to lower the programming barrier to constraint-based modelling. OptFlux [7] is a desktop application targeting strain optimisation workflows, offering a graphical interface to a curated set of simulation and evolutionary optimisation methods. Fluxer [6] provides a browser-based environment for interactive flux visualisation and basic FBA execution without requiring any local installation. CNApy [8] is an open-source desktop application written in Python that exposes a broader range of COBRA methods through a graphical front-end, including advanced techniques such as Minimal Cut Sets and network visualisation. COBRAxy [9] integrates constraint-based modelling into the Galaxy platform, enabling multi-sample flux analysis and visualisation through a browser-based workflow interface. Beyond simulation tools, Escher [21] provides an interactive web-based map for overlaying flux distributions onto curated metabolic pathway diagrams, and is widely used for result interpretation.

Despite this breadth of tooling, each platform addresses a specific subset of the GSM analysis pipeline in isolation. More accessible tools trade away analytical depth — flux sampling, transcriptomic integration, and advanced optimisation are typically absent — while more feature-complete environments continue to impose installation requirements or programming knowledge that limits their reach. Critically, none of these tools accepts a research objective stated in natural language and returns a coherent, multi-step executable workflow spanning the full pipeline.

2.3 Agentic AI and the ReAct Framework

The ReAct framework [22] introduced a paradigm for LLM-based agents that interleaves chain-of-thought reasoning with discrete tool invocations in an alternating Thought–Action–Observation loop. Rather than generating a response in a single forward pass, a ReAct agent explicitly reasons about what information or computation is needed, selects and calls an appropriate external tool, incorporates the returned observation into its reasoning context, and iterates until the task is resolved. This architecture enables LLMs to handle tasks that require multi-step reasoning grounded in external state — such as querying databases, executing code, or invoking domain-specific APIs — while maintaining an interpretable trace of the agent’s decision process. The Chat Mode of the system

developed in this thesis is built directly on the ReAct paradigm, with a suite of COBRApy-backed tools serving as the action space.

The broader applicability of agentic LLMs to biological data analysis has been demonstrated by tools such as BioOmni [15], AutoBA [16], and the Bioinformatics Agent (BIA) [17], each of which wraps domain-specific bioinformatics pipelines behind a natural language interface. These systems have shown that LLM agents can meaningfully lower the barrier to complex multi-omics analyses. Their scope, however, is oriented towards general bioinformatics workflows — sequence analysis, omics integration, and statistical processing — and does not extend to the constraint-based modelling methods that are central to GSM research. The system presented in this thesis addresses this gap by specialising the agentic architecture specifically for the GSM domain.

2.4 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) [23] addresses a fundamental limitation of parametric language models: the inability to reliably access knowledge that is absent from, or insufficiently represented in, training data. In the RAG paradigm, a retrieval component fetches relevant documents from an external knowledge base at inference time, and these documents are provided as context to the language model alongside the user query, grounding the generated response in verifiable external knowledge rather than implicit model weights. This approach has proven particularly effective in knowledge-intensive domains where factual precision and traceability are important.

The applicability of RAG extends naturally to planning tasks, where retrieved documents serve not merely as factual references but as structured exemplars that guide the construction of multi-step outputs. Lyu et al. [24] demonstrate this in their Retrieve-Plan-Generation framework, in which retrieval over a knowledge base of past instances directly informs the planning stage before generation, yielding more coherent and grounded outputs on knowledge-intensive tasks. The MetaPlan of the system developed in this thesis follows the same principle in the GSM domain: structured workflow steps extracted from published GSM literature are embedded into a persistent vector store, and at query time the most relevant steps are retrieved and provided to an LLM planner, which assembles them into a coherent, executable multi-step analysis plan. This grounds plan generation in real, published GSM practice rather than relying solely on the general reasoning capabilities of the underlying language model.

2.5 Research Gap

Despite the maturity of constraint-based modelling methods and the demonstrated success of ReAct-based agentic systems in adjacent biological domains, no existing tool supports end-to-end GSM analysis through a natural language interface, and no system grounds multi-step GSM workflow planning in structured knowledge retrieved from published literature. This thesis addresses both gaps; the design and implementation are described in the following chapter.

Section 3: Methods

3.1 System Architecture

The system is organised around two independent interaction modes — *MetaInteract* and *MetaPlan* — that share a common computational infrastructure. Both modes expose the same set of COBRApy analysis tools, operate on the same active metabolic model, and use the same LLM instance, but they differ fundamentally in how they orchestrate these resources to respond to a user query. Figure 3.1 shows the overall architecture of *MetaPilot*.

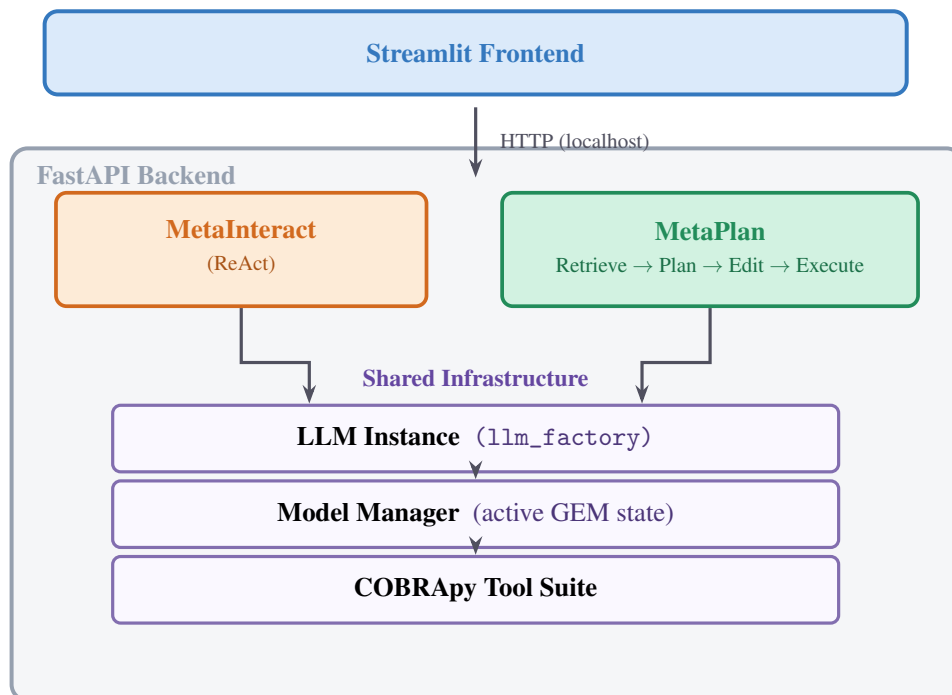


Figure 3.1: Overall *MetaPilot* system architecture.

The application follows a two-process design: a Streamlit frontend handles all user interaction and interface state, while a FastAPI backend handles all computation. The two processes communicate over a local HTTP interface. This separation ensures that the interface layer carries no scientific state — model data, tool outputs, and LLM context all live in the backend — which keeps the frontend simple and allows the backend to be developed and tested independently.

LLM integration via LlamaIndex. The backend builds on the LlamaIndex framework [25] as the primary abstraction layer for LLM interaction. LlamaIndex provides the agent runtime used in *MetaInteract*, the `FunctionTool` wrappers that expose COBRApy functions to the agent, and the chat interface used by the planner in *MetaPlan*. All LLM provider interactions are routed through a unified factory function, `get_llm(provider, model, api_key)`, which returns a LlamaIndex-compatible LLM object regardless of the underlying provider. Six providers are supported: Groq, OpenAI, Gemini, Ollama, Hugging Face Inference, and llama.cpp for locally-hosted GGUF models. The provider and model can be switched at runtime without restarting the application, making the system entirely model-agnostic. This is particularly relevant for deployment in environments where data privacy or offline operation is required, in which case a locally-hosted model can be substituted

with no changes to the rest of the system.

Model and session management. All metabolic models are managed through a `ModelManager` instance that is initialised once at server startup and shared across all endpoints. Models can be loaded either by providing a BioModels database identifier, which triggers an API fetch and in-memory parse, or by uploading an SBML file directly through the interface. The `ModelManager` tracks all loaded models and maintains the currently active model; all tools and the plan executor operate on this shared instance, so any state change made during a session — such as modifying reaction bounds or setting an objective — persists across subsequent tool calls regardless of which mode initiated them. Sessions are named by the user at startup, and a dedicated directory is created under `artifacts/<session_name>/` to store all file outputs generated during the session, including FVA result tables, knockout simulation summaries, and flux sampling matrices. At session end, the user may choose to retain or discard this directory.

3.2 COBRApy Tool Suite

Table 3.1: MetaPilot COBRApy tool suite organised by modelling lifecycle phase (38 tools)

Lifecycle Phase	Tool	Tool	Tool
<i>Model Acquisition & Inspection</i>	<code>load_model</code>	<code>check_model_loaded</code>	<code>get_current_model_id</code>
	<code>model_data</code>	<code>model_info</code>	<code>reaction_info</code>
	<code>metabolite_info</code>	<code>gene_info</code>	
<i>Model Reconstruction</i>	<code>build_model_with_carveme</code>	<code>build_context_model_with_corda</code>	<code>build_model_with_mackinac</code>
<i>Model Curation & Quality Control</i>	<code>run_memote_quality_report</code>	<code>check_model_consistency</code>	<code>check_mass_balance</code>
	<code>prune_unused_reactions</code>	<code>prune_unused_metabolites</code>	<code>find_blocked_reactions</code>
	<code>run_gap_filling</code>		
<i>Model Configuration & Editing</i>	<code>set_model_objective_value</code>	<code>set_reaction_bounds</code>	<code>add_reaction_to_model</code>
	<code>remove_genes</code>	<code>find_minimal_medium</code>	
<i>Flux Analysis & Simulation</i>	<code>run_flux_balance_analysis</code>	<code>run_parsimonious_fba</code>	<code>run_geometric_fba</code>
	<code>run_flux_variability_analysis</code>	<code>run_production_envelope</code>	<code>sample_metabolic_model</code>
<i>Perturbation & Knockout Analysis</i>	<code>gene_knockout_simulation</code>	<code>reaction_knockout_simulation</code>	<code>run_moma</code>
	<code>run_room</code>	<code>find_essential_genes</code>	<code>find_essential_reactions</code>
<i>Visualization</i>	<code>visualize_escher</code>	<code>list_escher_maps</code>	
<i>Utility</i>	<code>request_file_upload</code>		

The tool suite comprises 38 standalone Python functions, each wrapping one or more COBRApy operations and returning a uniform JSON-serialisable dictionary containing either a structured result payload or an error key. Entity-lookup tools (`reaction_info`, `metabolite_info`, `gene_info`) additionally apply fuzzy string matching via `SequenceMatcher` to resolve approximate or partial identifiers at query time. Every function carries a `_planner_meta` attribute — a dictionary specifying its canonical name, a one-line description, and a typed parameter schema (description, default value, and required flag per parameter) — which `tool_registry.py` harvests at import time to build three artefacts: `TOOL_FN_MAP` (name → callable, consumed by MetaPlan executor), `TOOL_USER_INPUTS` (parameter specifications rendered as UI input forms), and `AVAILABLE_TOOLS_TEXT` (injected verbatim into the LLM planner prompt). In Chat Mode the same functions are additionally registered as `LlamaIndex FunctionTool` objects, with function docstrings serving as the agent-facing description, so each tool requires no separate adaptation to serve both operating modes. Table 3.1 organises the

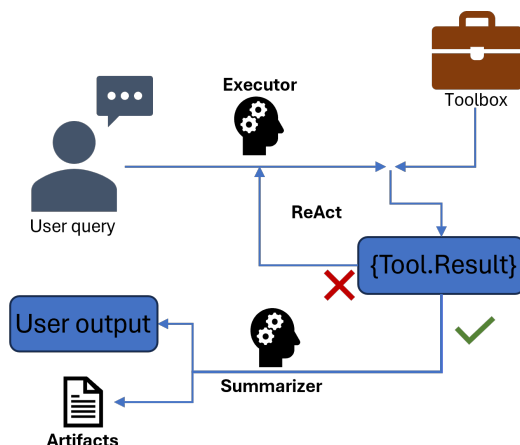


Figure 3.2: System architecture and multi-agent workflow in MetaInteract.

full suite by modelling lifecycle phase.

3.3 *MetaInteract*: Interacting with GSM Models Using Natural Language

MetaInteract provides a free-form natural language interface to the COBRApy tool suite. The user submits a query in plain English — for example, requesting a gene knockout study, asking for an interpretation of FVA results, or querying model structure — and the system autonomously determines which tools to invoke and in what order to produce a complete response. MetaInteract is built on the ReActAgent provided by LlamaIndex, which implements the Reasoning and Acting (ReAct) framework [22]. The full pipeline for a single query is shown in Figure 3.2.

Agent construction. The Executor Agent is initialised by registering the complete COBRApy tool suite as `FunctionTool` objects, each wrapping a tool function with a natural-language name and description that the agent uses to reason about when and how to invoke it. The agent is constructed fresh each time the LLM provider is changed, ensuring it always holds a reference to the current LLM instance. The full system prompt passed to the agent is composed of two parts: a role and responsibility prompt and a context prompt. The role prompt establishes the agent’s persona as a scientific assistant for biologists without programming expertise, defines its core responsibilities — model loading, simulation, result interpretation, and error handling — and sets behavioural constraints such as never hallucinating tool outputs and always presenting results in biological rather than programming terms. The context prompt enumerates the available tools and their usage, describes the structure of COBRApy models (reactions, metabolites, genes, compartments), and specifies session continuity rules such as persistence of flux bounds and objective settings across queries.

Two-stage response pipeline. A design challenge in deploying a ReAct agent for scientific users is that the Executor Agent’s raw output — which interleaves reasoning traces, tool call records, and partial results — is not suitable for direct presentation. To address this, a Summarizer Agent makes a dedicated second LLM call after the Executor Agent completes its reasoning cycle. The Executor Agent’s final response is passed to the same LLM with a dedicated reformatting prompt

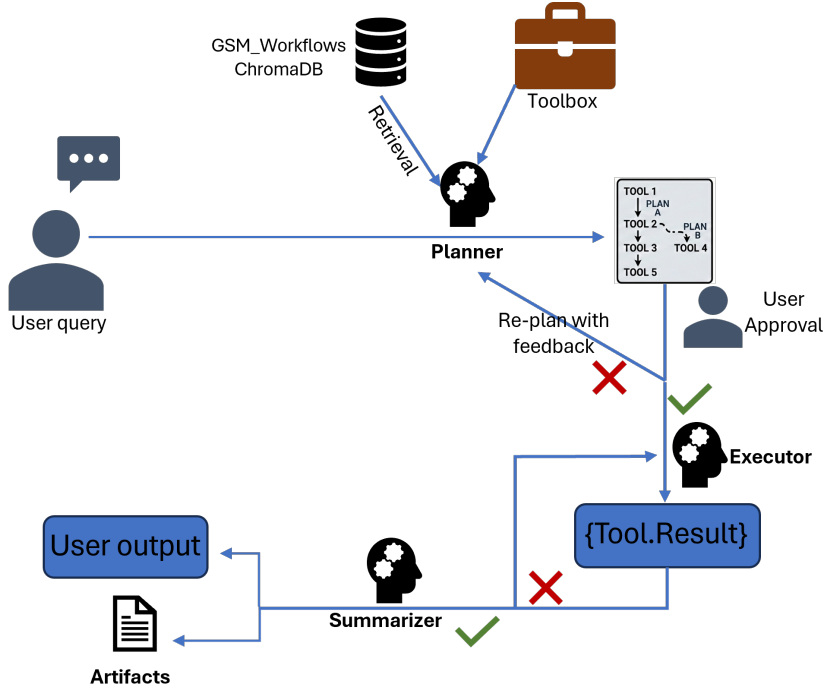


Figure 3.3: System architecture and multi-agent workflow in MetaPlan.

and a separate system prompt that instructs the model to preserve all data from the agent response without omission, render any tabular data in structured table format, maintain a formal scientific tone appropriate for graduate-level biology researchers, and suppress any programming artefacts or internal reasoning that should not be visible to the user. This two-stage design separates the concern of *what to compute* (the Executor Agent) from the concern of *how to present it* (the Summarizer Agent), resulting in responses that are both computationally grounded and scientifically accessible to the intended user.

3.4 *MetaPlan*: Multi-Step GSM Workflow Planning and Execution

MetaPlan is designed for users who want to execute a structured, multi-step GSM analysis grounded in published methodology rather than issuing free-form queries. Instead of a reactive loop, MetaPlan follows a plan-and-execute architecture: the system first retrieves relevant workflows from the Procedural Knowledge Base (PKB), generates a complete analysis plan, presents it to the user for review and editing, and then executes the approved steps one at a time. Three components drive this pipeline — a Planner Agent that generates the plan, an Executor Agent that runs the tools, and a Summarizer Agent that interprets each result for the user — and the full pipeline is shown in Figure 3.3.

Workflow retrieval. Before any plan is generated, the system retrieves methodologically relevant workflows from the PKB using GSMRetriever. The user’s query is embedded using the BAAI/bge-large-en-v1.5 model with an asymmetric BGE query prefix and compared against the ChromaDB collection via a two-phase search. In Phase 1, the query is matched against tier 0 (paper-level) and tier 1 (pipeline category-level) entries to identify broadly relevant papers and pipeline stages. In Phase 2, the query is matched against tier 2 (sub-workflow-level) and tier 3

(individual step-level) entries to retrieve granular workflow candidates. Tier 2 hits are the primary plan seeds: each carries a complete list of step identifiers corresponding to a validated sub-workflow from a published paper, ensuring that entire methodological sequences can be reconstructed even when individual intermediate steps do not score highly against the query on their own. Tier 3 hits provide supplementary individual steps that can fill gaps not covered by any sub-workflow.

Plan generation and review. The retrieval results are passed to the Planner Agent, implemented as `GSMPanner`, which makes a single LLM call at temperature 0.0 to produce a structured, deterministic plan. The planning prompt presents the top-scored sub-workflows and supplementary steps as candidates, alongside the list of available COBRAPy tools with their exact names, and instructs the LLM to select, order, and justify a set of steps that address the user’s query using only the provided candidates. The output is a validated JSON plan in which each step carries a tool name, a rationale, and a set of user-configurable parameters. Once generated, the plan is presented to the user through an interactive editor: individual steps can be included or excluded, parameter values can be filled in through guided input fields, and new steps can be inserted between existing ones. The user may also provide free-text feedback to request changes to the plan, which triggers a revision call to the Planner Agent that updates only the requested elements while preserving the rest of the plan unchanged.

Plan execution. Once the user approves the plan, the Executor Agent runs each included step in sequence by dispatching directly to the corresponding tool function via a fixed function map, bypassing the ReAct agent entirely. This direct dispatch is deliberate: since the plan already specifies exactly which tool to call with which parameters, introducing an LLM reasoning loop at execution time would add non-determinism and latency without any benefit. After each step completes, the Summarizer Agent makes a dedicated LLM call using a step interpretation prompt, which produces a concise, user-facing summary of the step outcome — reporting whether it succeeded or failed, and describing the result in plain scientific language. The per-step summaries are accumulated across the execution sequence and appended to the session’s chat history upon completion.

3.5 PKB Construction Pipeline

3.5.1 Corpus Collection

The PKB construction pipeline operates on a curated corpus of published GSM literature, assembled through citation-graph traversal. The collection strategy began with a seed set of highly-cited GSM papers. For each seed node, all papers citing it were identified via their DOIs, and this process was applied iteratively, expanding the collection outward through the citation network while remaining anchored to the core GSM literature. Papers were collected from six sources using publisher-specific access channels, as summarised in Table 3.2.

Papers were received in three formats depending on the source: PDF (Springer, Nature, Wiley, Elsevier, Unpaywall, and non-OA PMC scrape), JATS XML (PMC FTP), and non-JATS XML (a small subset of programmatically retrieved papers). All PDF and JATS XML documents were processed using Docling [26] Papers arriving in non-JATS XML formats were handled through a dedicated programmatic parser.

Table 3.2: GSM literature corpus: coverage by publisher and access method.

Publisher	Access Method	Supplementary Files
PMC (open access)	PMC FTP bulk download	Yes
PMC (non-OA)	Playwright web scraping	Yes
Springer	Playwright web scraping	Yes
Nature	Playwright web scraping	Yes
Wiley	Wiley Full-Text API	No
Elsevier	Elsevier TDM API	No
Unpaywall	Unpaywall API (OA links)	Partial

3.5.2 Corpus Filtering

The raw corpus assembled contained a heterogeneous mix of document types including review articles, meta-analyses, perspective pieces, and editorials. Additionally, the citation-graph expansion strategy can admit papers that cite foundational GSM works in passing without themselves constituting GSM studies. Two sequential filters were applied to address both problems.

Filter 1: Review Paper Removal via Section Detection

The first filter operates on the document’s heading structure where a paper is retained as a non-review empirical study if and only if it satisfies the following condition:

$$\text{has_methods} \wedge (\text{has_results} \vee \text{has_discussion}) \quad (3.1)$$

with the rationale that review papers, perspective articles, and editorials lack one or both of these structural elements.

Filter 2: GSM Relevance Classification

Papers passing the structural filter are further classified for genome-scale metabolic modelling relevance using **Gemini-3.5-Flash**. The abstract and extracted methods section are submitted to the model, which returns a binary Yes/No decision via API-enforced JSON structured output.

3.5.3 Methods Extraction and Step Decomposition

This sub-section describes how the methods text is isolated from each paper and then decomposed into a structured, per-step JSON representation by a language model.

Stage 1: Methods Text Extraction

Each paper’s full text, produced by Docling in markdown format as described in Section 3.5.1, is passed through a custom multi-format section extractor that scans the document for a Methods section heading and collects all text from that heading until a terminal section heading is encountered. Terminal headings include Results, Discussion, Conclusions, Acknowledgements, References, and related variants. Before the extracted text undergoes preprocessing before it is passed to the language model.

Stage 2: Workflow Step Decomposition via Gemini-3.5-Flash

The pre-processed methods text for each paper is passed to **Gemini-3.5-Flash** via the Google Generative AI API, which decomposes the methodology into a structured list of individual workflow steps. The model is instructed through a two-shot prompt that supplies two complete example papers alongside their ground-truth step extractions. The prompt instructs the model to extract only steps that are directly related to constraint-based metabolic modelling. The generation temperature is set to 0.0 to maximise output determinism.

Table 3.3: Fields of the ModelingStep schema used for step extraction.

Field	Description
step_id	Sequential identifier of the step within the paper
step_category	One of seven workflow categories (see text)
step_name	Short descriptive name of the step
objective	Detailed objective: what is achieved and why it matters for subsequent steps
inputs	Exhaustive list of non-GSM inputs consumed
outputs	Exhaustive list of non-GSM outputs produced
GSM_inputs	GSM model state consumed: NA, Draft_GSM, or Curated_GSM
GSM_outputs	GSM model state produced: NA, Draft_GSM, or Curated_GSM
technique	Specific tool, algorithm, or method used (one per step)
text_span	Verbatim sentences from the source methods text that describe this step

The ExtractionResult schema defines a list of ModelingStep records, each carrying the fields described in Table 3.3. Of particular note is the step_category field. The step_category field classifies each step into one of seven mutually exclusive types, defined in Table 4.7.

Table 3.4: The seven step_category values and their operational definitions.

Category	Definition
Model Reconstruction	Import or build the metabolic network — defining reactions, metabolites, stoichiometry
Model Curation	Permanent structural edits to the model: adding or removing reactions, gap-filling
Model Validation	Test model quality and behaviour against experimental data
Constraint Definition & Config.	Simulation-specific, non-structural setup: setting flux bounds, defining media, applying gene knockouts
Optimization	Algorithmic solving for flux distributions or objective values
Sensitivity Analysis	Robustness and variability assessment via FVA, gene or reaction essentiality screens
Visualization	Generation of plots, flux maps, or graphical outputs of model structure or simulation results

3.5.4 Objective Hierarchy Generation

The step extraction stage produces a structured JSON for each paper containing a list of `ModelingStep` records. Each step carries an objective field written during extraction, but this field is extraction-oriented. For RAG, the indexed text must instead be written in the vocabulary a researcher would use when formulating a query. Furthermore, a flat list of step-level entries is insufficient for queries that operate at different granularities. To address both problems, each paper’s step list is enriched with a

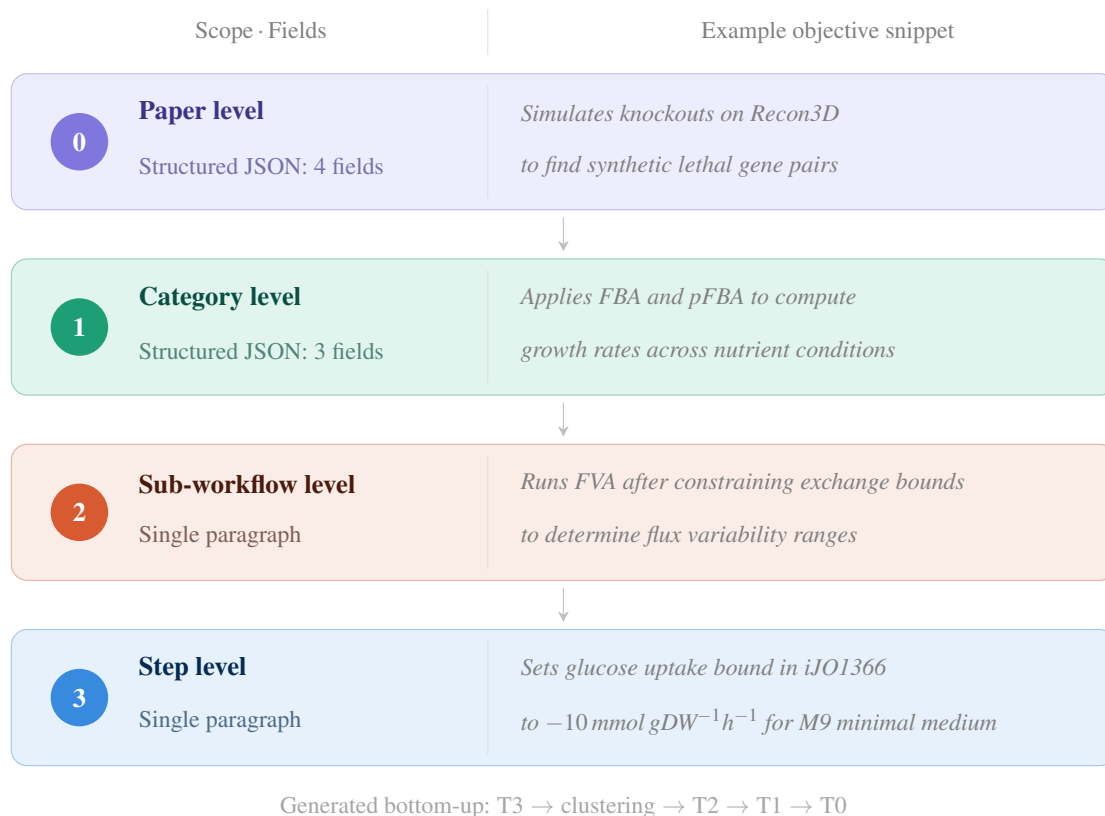


Figure 3.4: The four-tier objective hierarchy...

four-tier objective hierarchy. The four tiers and their scope are summarised in Table ?? . Objectives are generated bottom-up in the order $T3 \rightarrow \text{clustering} \rightarrow T2 \rightarrow T1 \rightarrow T0$, so that each tier can draw on the richer, more grounded language of the tiers below it. All four tiers are generated using a locally-run reasoning model served via `llama.cpp` [27]. **Qwen3.5-9B** was the model used, with `temperature=0`.

Tier 3: Step-Level Objectives

Tier 3 objectives are generated in a single batched JSON call that presents all steps from a paper to the model simultaneously, alongside the full methods text as context.

Sub-Workflow Clustering

Before Tier 2 objectives can be generated, the steps of each paper must be grouped into meaningful sub-workflow clusters. This is done through an LLM-based sub-trajectory identification step that operates on the Tier 3 objectives generated above.

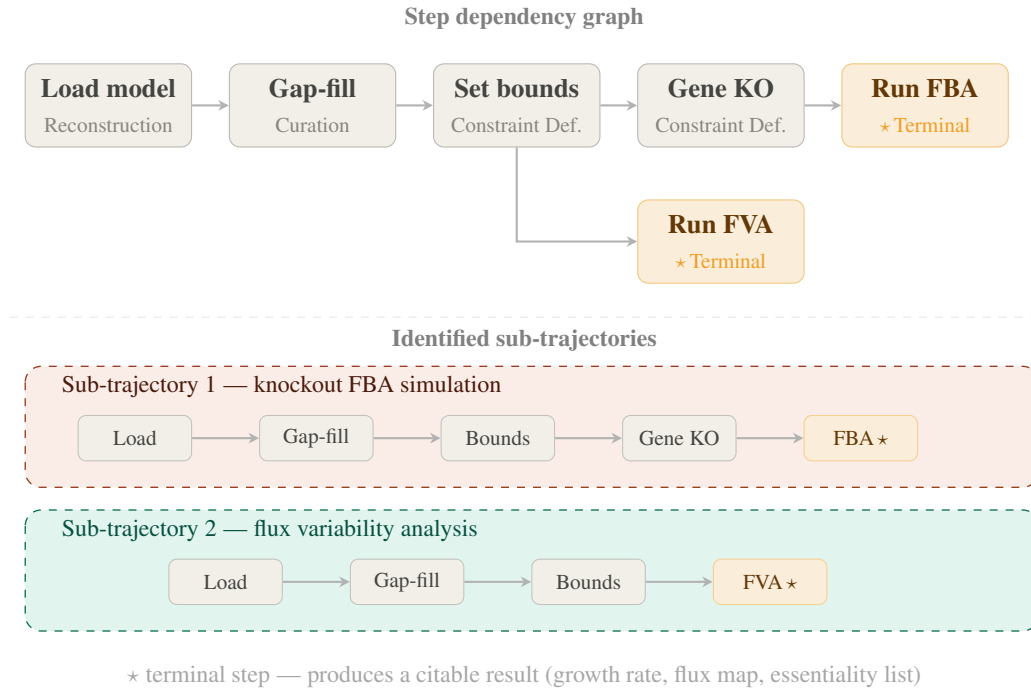


Figure 3.5: Clustering into Tier 2 sub-workflows

The model is presented with all Tier 3 objectives alongside their source text spans and prompted to first identify all *terminal steps*: steps that produce a result a researcher would cite or report, such as a growth rate prediction or a set of strain engineering targets. For each terminal step, the model then identifies the minimal set of preceding steps that are causally necessary to reach it, tracing dependencies purely through each step’s objective. Each identified group—comprising the terminal step and its minimal prerequisite steps—constitutes one sub-trajectory cluster.

Tier 2: Sub-Workflow Objectives

One Tier 2 objective is generated per sub-trajectory cluster. The model is presented with the cluster’s terminal step and its Tier 3 objective, the Tier 3 objectives of all prerequisite steps, and the full methods text. The objective is written with terminal-step anchoring to ensure that Tier 2 objectives are discriminative across clusters within the same paper rather than converging on generic setup descriptions.

Tier 1: Category-Level Objectives

One Tier 1 objective is generated per step category present in the paper. The model is presented with either the Tier 2 group objectives for that category (if sub-groups were identified) or the Tier 3 step objectives directly (if the category had fewer steps than the clustering threshold), alongside the full methods text. The Tier 1 objective is returned as a structured JSON object with three fields: `phase_role` (a single sentence describing the pipeline stage), `operations` (one to two sentences enumerating the specific techniques, algorithms, and tools present in this category as drawn from the Tier 2 or Tier 3 objectives), and `relevance` (two to three sentences written in researcher-query language that a researcher would use when searching for steps of this type, naming concrete example

tasks). The `relevance` field is the most retrieval-critical and is the only field embedded into the vector index at this tier.

Tier 0: Paper-Level Objective

One Tier 0 objective is generated for the paper as a whole. The model receives all Tier 1 objectives alongside the full methods text and produces a structured JSON with four fields: `research_question` (the biological or computational question the paper addresses), `organism_context` (the organism, cell type, tissue, or biological system studied), `pipeline_summary` (two sentences covering the full computational pipeline from first stage to final output, naming every major technique and the key data inputs and outputs), and `relevance` (three to four sentences in researcher-query language synthesising across all pipeline stages and naming specific techniques, tools, model types, and analysis types). The `relevance` field is written to be inclusive: a researcher reading it alone should be able to judge whether the paper’s pipeline is relevant to their specific task.

3.5.5 Vector Index Construction

The final stage of the PKB pipeline converts the four-tier objective hierarchy for each paper into a persistent vector index that the MetaPlan retrieval module can query at runtime. All tiers from all papers are indexed into a single ChromaDB collection named `gsm_workflows`, using **BAAI/bge-large-en-v1.5** as the embedding model, accessed via the `sentence-transformers` library.

Asymmetric Embedding with BGE Prefix Prompting

BAAI/bge-large-en-v1.5 supports asymmetric retrieval through prefix strings that signal to the model whether the text being encoded is a document or a query. Two distinct prefixes are used: documents are prepended with “*Represent this document for retrieval:*” at index time, and user queries are prepended with “*Represent this query for retrieval:*” at search time. Using the correct prefix at each stage is required for optimal retrieval quality. Both prefix constants are exported from the indexing module and imported directly by the retrieval module to guarantee consistency. All embeddings are L2-normalised before storage, so cosine similarity at query time reduces to an efficient dot product.

Per-Tier Embedding Strategy

Not all fields of the four-tier objective are equally useful for retrieval, and embedding irrelevant text would dilute the semantic signal. The text embedded for each tier is therefore selected to maximise the match between what a researcher would type as a query and what is stored in the index. (see Table: 3.5)

Metadata Schema and Hard Filtering

Each entry in the collection carries a metadata dictionary that enables hard filtering at query time, independently of semantic similarity. The metadata fields are described in Table 3.6. The `tier` field allows the retrieval module to restrict searches to a specific level of granularity. The `step_category` and `technique` fields (populated for Tier 1–3 entries) support category- or tool-specific filtering before semantic ranking. The `step_ids` field, stored as a JSON-encoded list, maps each Tier 2 or

Table 3.5: Per-tier embedding strategy: text indexed and rationale for field selection.

Tier	Embedded fields	Rationale
0	relevance + research_question	relevance is written in researcher-query language and provides the primary retrieval signal; research_question adds biological and organism-level context that helps broad queries match the right papers
1	relevance only	phase_role and operations are descriptive rather than query-facing; including them would dilute the retrieval signal
2	Full objective paragraph	Tier 2 objectives are already concise and terminal-step-anchored, so the complete paragraph is the appropriate retrieval unit
3	Full objective paragraph	Step-level objectives are written in action-oriented, domain-rich language designed to match specific researcher queries

Tier 3 entry back to the specific step identifiers it covers, allowing the retrieval module to recover the full step records from the source JSON after a query.

Table 3.6: Metadata fields stored per entry in the gsm_workflows ChromaDB collection.

Field	Type	Description
tier	int	Hierarchy level: 0 (paper), 1 (category), 2 (sub-workflow), 3 (step)
paper_id	str	Identifier of the source paper, derived from the filename stem
entry_id	str	Unique entry identifier within the paper (e.g. T0, T1_Optimization, T3_S4)
step_ids	str	JSON-encoded list of step IDs covered by this entry; empty list for Tier 0/1
step_category	str	Pipeline category name for Tier 1–3 entries; empty for Tier 0
technique	str	Specific tool or algorithm name for Tier 3 entries; empty otherwise

Section 4: Results

4.1 A Comparison of *MetaPilot* with Existing Tools

Table 4.1 situates the proposed system within the landscape of existing GSM analysis tools and LLM-based biological workflow agents. Four established GSM platforms are included — OptFlux [7], Fluxer [6], CNApy [8], and COBRAxy [9] — alongside BioOmni [15], a general-purpose biomedical LLM agent that includes a small set of metabolic modelling tools. The comparison covers seven criteria: interface type, deployment model, GSM tool coverage, multi-step workflow automation, literature-grounded plan generation, user-editability of generated plans, and agentic execution.

Several observations follow from the comparison. First, the proposed system and BioOmni are

Table 4.1: Comparison of the proposed system with existing GSM analysis platforms and LLM-based agents. ✓ = fully supported; Partial = limited or template-based support; — = not supported.

Criterion	MetaPilot	BioOmni	OptFlux	Fluxer	CNApy	COBRAxy
Interface type	Nat. lang.	Nat. lang.	GUI	GUI	GUI	GUI
Deployment	Web	Web	Desktop	Web	Desktop	Web
GSM tool coverage	End-to-end GSM modelling life-cycle support	FBA, metabolic perturbation simulation	FBA, MFA, EFMs, strain design	FBA, gene knockout, visualisation	FBA, FVA, MCS, EFMs, strain design, visualisation	FBA, flux sampling, omics integration, visualisation
Multi-step automation	✓	✓	Partial	—	Partial	Partial
Lit.-grounded planning	✓	Partial	—	—	—	—
User-editable plan	✓	—	—	—	—	—
Agentic execution	✓	✓	—	—	—	—

the only two tools offering a natural language interface accessible through a web browser without any local installation. All four GSM-specific platforms require either a desktop installation or access to a Galaxy server, and none accepts free-form natural language input. Second, while GUI-based tools such as CNApy and COBRAxy support partial automation through fixed analysis templates or platform-level workflow chaining, none of them dynamically composes an analysis plan in response to a user-stated research objective. Third, BioOmni, despite being a capable general-purpose agent, includes only two metabolic modelling tools — FBA and kinetic perturbation simulation — and its retrieval-augmented planning draws on a broad biomedical literature base with no GSM-specific grounding. In contrast, the proposed system’s MetaPlan retrieves structured workflows from a knowledge base built exclusively from 3,878 curated GSM papers, producing plans that reflect the actual methodological patterns of the GSM literature. Finally, the user-editable plan — in which the system presents a literature-grounded, step-by-step workflow for the user to review, modify, and approve before execution — is a capability unique to this work among all compared tools.

4.2 Case Studies

All case studies that follow used the gemini-3.1-pro-preview model.

4.2.1 MetaInteract: Agentic Chat Mode

MetaInteract is the agentic chat interface of the system, enabling users to interact with genome-scale metabolic models through free-form natural language. Queries are processed by a ReAct-based LLM agent (Gemini 3.1 Pro Preview) that reasons over the available COBRApy-backed tool suite and invokes the appropriate tool for each request. The following two case studies illustrate representative interactions spanning model reconstruction, quality assessment, and gene essentiality analysis.

Case Study 1: *De Novo* Model Reconstruction and Quality Assessment

This case study demonstrates the system’s ability to handle a model request that cannot be fulfilled from an existing database (BiGG, BioModels) and to guide the user to an automated *de novo*

reconstruction workflow. The interaction is conducted in three turns, each invoking a distinct tool.

Turn 1 — Model retrieval attempt. The user issues the query “*get me the mycoplasma genitalium model.*” The agent searches the BiGG and BioModels databases but finds no pre-existing model for *Mycoplasma genitalium*. Rather than returning a bare error, it presents two alternative reconstruction pathways: (i) automated draft reconstruction from a PATRIC genome identifier using the `build_modelseed_model` tool, or (ii) homology-based reconstruction from a protein FASTA file using the `run_carveme` tool.

Turn 2 — CarveMe reconstruction. The user attaches the proteome FASTA file `M_genitalium_G37.faa` and requests reconstruction via CarveMe. The agent invokes `run_carveme`, which completes successfully and returns a structured summary of the reconstructed model. The resulting model, assigned the identifier `M_genitalium_G37`, comprises 871 reactions, 653 metabolites, and 130 genes, and is saved as an SBML file at `artifacts/case_1.1/outputs/carveme/M_genitalium_G37.xml`.

Turn 3 — Memote quality assessment. The user then requests “*run memote on the model.*” The agent invokes the `memote_report` tool, which executes the Memote test suite against the reconstructed model and returns a structured evaluation summary. The model achieves an overall Memote score of 87.3% across 74 tests (38 passed, 36 failed, 3 skipped). Despite this score, Memote reports an overall status of *Failed*, which is consistent with the expected behaviour of automated CarveMe reconstructions: Memote’s test suite includes strict annotation-completeness checks that freshly reconstructed models routinely fail in the absence of manual curation. The 87.3% score nonetheless indicates strong metabolic consistency and structural integrity of the model, confirming its suitability for downstream constraint-based analysis.

Case Study 2: Gene Essentiality Analysis in *Helicobacter pylori*

This case study demonstrates the system’s ability to load a pre-existing curated model and perform a gene knockout simulation from a natural language query, including resolution of a common gene name to its model-internal locus tag identifier.

Turn 1 — Model loading. The user issues the query “*load the helicobacter model.*” The agent identifies and loads the *Helicobacter pylori* genome-scale metabolic model `iIT341` [28], confirming successful initialisation.

Turn 2 — Gene knockout simulation. The user then asks “*what is the effect of the knockout of the murB gene on this organism’s growth?*” The agent correctly resolves the common gene name *murB* to the model’s internal locus tag `HP1418` and invokes `gene_knockout_simulation`. The simulation returns a post-knockout growth rate of 0.0 (solver status: optimal), which the agent interprets as indicating that *murB* is essential for the organism’s viability under the simulated growth conditions. This result is biologically consistent: *murB* encodes UDP-*N*-acetylenolpyruvoylglucosamine reductase, a key enzyme in the peptidoglycan biosynthesis pathway responsible for bacterial cell wall integrity, and its deletion is expected to be lethal in *H. pylori*.

4.2.2 MetaPlan: Multi-step plan mode

Case Study: Breast Cancer Context-Specific Modelling and HK2 Drug Simulation

User Query:

“I want to create a model for a breast cancer cell and then study the growth of the cell after simulating the effects of a drug targeting the gene HK2.”

Overview. This case study exercises the full MetaPlan pipeline: context-specific model reconstruction followed by drug simulation via gene knockout. The GSMPlanner assembled a six-step workflow (Table 4.2), drawing from three literature sources, which the executor ran deterministically to completion.

Table 4.2: Six-step plan generated by GSMPlanner for the breast cancer HK2 drug simulation case study, with execution outcomes.

Step	Name	Tool	Status
1	Load Generic Human Metabolic Model	load_model	✓
2	Reconstruct Breast Cancer Model	build_context_model_with_corda	✓
3	Simulate Wild-Type Growth	run_fba	✓
4	Visualise Wild-Type Fluxes	visualize_escher	✓ [†]
5	Simulate HK2 Drug Targeting	gene_knockout_simulation	✓
6	Visualise Knockout Fluxes	visualize_escher	✓

[†] Succeeded after 2 automatic retry attempts.

Execution. Recon3D (10,600 reactions) was loaded and passed to `build_context_model_with_corda` with breast cancer transcriptomic data (`brca_corda_input.csv`). CORDA pruned the reference model to a context-specific network of 4,129 reactions, 1,439 genes, and 2,963 metabolites, with 81.9% of model genes successfully mapped to the expression dataset. FBA on the reconstructed model returned an optimal wild-type growth objective of 202.0867. The wild-type flux distribution was visualised on the `RECON1.Carbohydrate metabolism` Escher map — recovered autonomously after two executor retries — and the *HK2* gene (locus tag 3099_AT1) was then knocked out to simulate drug targeting.

Result. The post-knockout growth objective remained **202.0867** — identical to the wild-type — indicating that the reconstructed breast cancer model is metabolically robust to single-gene deletion of *HK2*. The Plan Complete screen summarised this as:

Targeting the HK2 gene has no effect on the growth rate of the simulated breast cancer cell. The context-specific breast cancer model was successfully reconstructed from the Recon3D base model, yielding a specialised network of 4,129 reactions. Baseline simulations established an optimal wild-type growth objective value of 202.0867. Following the simulated drug knockout of the HK2 gene (ID 3099_AT1), the cell’s growth rate remained entirely unchanged at 202.0867, indicating metabolic redundancy or alternative pathways compensating for the loss.

Discussion. The case study validates three MetaPlan properties. The planner correctly sequenced reconstruction before simulation and sourced steps from multiple papers without manual intervention. The executor demonstrated self-recovery by autonomously retrying a failed Escher map-name resolution. The result itself — no growth change upon *HK2* deletion — is a biologically valid prediction reflecting the metabolic redundancy of alternative hexokinase isozymes retained in the CORDA model, and is communicated directly to the user in the Plan Complete summary.

4.3 Retrieval Quality

The GSMRetriever was evaluated on a manually constructed set of 20 natural language queries spanning all seven GSM step categories and three complexity levels — simple, multi-step, and abstract — reflecting the range of queries a practitioner would realistically pose to the system. Full details of the study design and annotation protocol are published in the GitHub repositories 8. The distribution of queries is shown in Table 4.3 and the results in Table 4.4.

Table 4.3: Distribution of the 20 evaluation queries by step category and complexity.

Step Category	Total	Simple	Multi-step	Abstract
Model Reconstruction	3	1	2	0
Model Curation	2	1	1	0
Model Validation	3	1	1	1
Constraint Def. & Configuration	2	1	1	0
Optimization	4	1	2	1
Sensitivity Analysis	4	1	2	1
Visualisation	2	1	1	0
Total	20	7	10	3

The system achieved an overall HR@3 of 0.700, returning at least one relevant sub-workflow in 14 of the 20 queries. The MRR of 0.467 indicates that when a relevant result is present it typically appears near rank 1. These figures are consistent with what one would expect from a semantic retrieval system built on heterogeneous paper extractions without query-document supervision or embedding fine-tuning, and indicate that the PKB’s semantic structure is sufficiently coherent to surface methodologically relevant workflows for the majority of realistic research queries. Abstract queries produced the highest P@3 (0.500) and MRR (0.500), suggesting that richly contextualised queries provide stronger semantic signal for matching against sub-workflow objectives than brief single-objective queries.

Table 4.4: Retrieval quality metrics overall and by query complexity. P@3 weights Relevant as 1.0 and Partial as 0.5; HR@3 indicates whether at least one Relevant result appeared in the top 3; MRR is the mean reciprocal rank of the first Relevant result.

Query Type	n	P@3	HR@3	MRR
Simple	7	0.429	0.714	0.476
Multi-step	10	0.483	0.700	0.450
Abstract	3	0.500	0.667	0.500
Overall	20	0.483	0.700	0.467

The strongest individual results were Q13 (drug target prioritisation and minimal genome design; P@3 = 0.83) and Q18 (diagnosing a model that disagrees with experimental phenotypes; P@3 = 0.83). The clearest failure was Q15, a simple visualisation query that returned no relevant results (HR@3 = 0). Visualisation is the most sparsely documented step type in the PKB — most GSM papers describe simulation results numerically rather than detailing the visualisation procedure — and this sparsity limits the retriever’s coverage for visualisation-centric queries. This limitation is discussed further in Section 7.

A category precision metric was also computed but is not reported as a primary result. Real GSM workflows are inherently multi-category, and a query about strain engineering legitimately retrieves Optimization, Constraint Definition, and Model Curation steps simultaneously. Penalising this cross-category retrieval conflates semantic relevance with categorical narrowness and is not an appropriate measure of retrieval utility for this system. The evaluation was conducted by the author and constitutes an indicative rather than definitive benchmark.

4.4 PKB Pipeline

Corpus Statistics

The PKB construction pipeline was applied to the Research Literature Corpus (RLC), a large-scale collection of GSM-related papers assembled via citation-graph traversal from seven sources: PMC FTP downloads, PMC web scraping, the Wiley API, the Elsevier TDM API, Playwright-based scraping of Springer and Nature, and the Unpaywall API for open-access papers. In total, the RLC comprises **14,397 papers** amounting to approximately 237.3 million tokens. A per-source breakdown is given in Table 4.5.

PMC is the dominant source, together contributing 8,636 papers (60%) and the largest share of tokens. On average, each paper in the corpus spans approximately 16,486 tokens with a mean sentence length of 30.34 tokens, characteristic of the verbose, protocol-heavy writing style of GSM literature. In addition to manuscript text, the corpus includes 81,738 inline figures and 443,746 supplementary files, covering model files, flux distribution tables, and reaction network data — resources that could further enrich future pre-training signals.

Table 4.5: Research Literature Corpus statistics by source.

Publisher	# Papers	# Tokens	# Images	# Suppl. Files
PMC FTP	7,028	94,596,376	35,021	426,215
PMC Scrape	1,608	31,796,234	9,848	13,470
Wiley	1,173	22,408,859	8,852	—
Springer	1,240	21,294,456	7,240	4,061
Elsevier	838	9,875,832	—	—
Nature	14	126,142	70	—
Unpaywall	2,496	57,251,558	20,707	—
Total	14,397	237,349,457	81,738	443,746

Corpus Filtering

Not all 14,397 papers in the RLC are suitable for PKB extraction. Review articles, perspective pieces, and papers only incidentally related to genome-scale metabolic modelling must be excluded before any LLM processing is attempted. The pipeline applies a two-stage filtering step to address this.

The first filter operates on document structure. A lightweight section detector scans each paper’s converted markdown text and checks for the presence of a Methods section alongside at least one of a Results or Discussion section. Papers failing this structural check are classified as non-empirical and excluded. Of the 14,397 papers, 4,047 were removed at this stage, leaving 10,350 empirical papers.

The second filter addresses GSM relevance. Citation-graph traversal can surface papers from adjacent fields — systems biology, general bioinformatics, metabolomics — that are only incidentally related to constraint-based metabolic modelling. Each remaining paper’s abstract is submitted to Gemini-3.5-Flash with a binary classification prompt; only papers receiving a Yes response are carried forward. This reduced the pool to 4,316 papers. A final deduplication step removes papers appearing under multiple publishers or access routes, bringing the final count to **3,878 papers** on which PKB extraction is performed. The full filtering funnel is summarised in Table 4.6.

Table 4.6: PKB pipeline corpus filtering funnel.

Stage	Papers
Total papers in corpus (RLC)	14,397
Failed review check (excluded)	4,047
Passed review check	10,350
Found to be GSM-related	4,316
After deduplication	3,878

Step Extraction and Category Distribution

Each of the 3,878 papers was processed through Stage 2 of the pipeline, in which Gemini-3.5-Flash decomposes the extracted methods text into a sequence of typed workflow steps. In total, the pipeline extracted **27,593** steps across all papers, with an average of **7.1** steps per paper. Every step is assigned to one of seven mutually exclusive categories reflecting its functional role within a GSM workflow. The distribution across these categories is shown in Table 4.7.

The distribution reflects the structure of published GSM workflows: Optimization steps (primarily FBA and its variants) and Constraint Definition & Configuration steps together constitute the

Table 4.7: Distribution of extracted PKB steps by category.

Step Category	Step Count	Percentage (%)
Optimization	5254	19
Constraint Definition & Configuration	5567	20
Model Curation	6570	24
Model Reconstruction	3528	13
Model Validation	1455	5
Sensitivity Analysis	4542	16
Visualisation	677	3
Total	27,593	100.0

largest share of extracted knowledge, consistent with the centrality of flux balance analysis in the GSM literature. Model Reconstruction and Curation steps are also well-represented, capturing the substantial body of work on automated and semi-automated GEM assembly. Sensitivity Analysis and Visualisation steps are comparatively less frequent, reflecting that these are downstream activities not present in every published workflow.

Knowledge Base Scale

Following step extraction, Stage 3 of the pipeline enriches each step-level JSON with a four-tier objective hierarchy and embeds all entries into a ChromaDB persistent vector store using the BAAI/bge-large-en-v1.5 model (1,024-dimensional embeddings). The resulting `gsm_workflows` collection contains entries at four levels of granularity — paper (Tier 0), pipeline category (Tier 1), sub-workflow (Tier 2), and individual step (Tier 3) — with a total of **58,547** indexed vectors. This tiered structure enables the GSMRetriever to recover both fine-grained individual steps and complete sub-workflow clusters in a single two-phase retrieval pass, as described in Section 3.4.

Section 5: Future Work

Several directions remain open for extending the system presented in this thesis.

The most immediate opportunity lies in expanding the tool suite. On the GSM side, the COBRA Toolbox for MATLAB supports a significantly wider range of genome-scale methods — including strain design algorithms, transcriptomic data integration, and thermodynamic flux analysis — that have no direct COBRAPy equivalents. Beyond GSM-specific tools, the agent could also be equipped with access to external resources such as live literature search via APIs like PubMed or Semantic Scholar, allowing the system to handle queries that fall outside the static knowledge base and to ground its responses in up-to-date published evidence.

A second direction is the fine-tuning of the models that underpin the system. The embedding model currently used for retrieval is a general-purpose encoder; a model fine-tuned on GSM literature would better capture domain-specific vocabulary and the semantic relationships between workflow concepts, improving retrieval quality. More broadly, all agent components in the current system — the planner, the executor, and the chat agent — rely on general-purpose LLMs. Domain-adapted models trained on GSM and constraint-based modelling literature would likely improve the accuracy and reliability of plan generation and tool invocation in this specialised setting.

Finally, the agentic capabilities of the system can be strengthened by replacing or augmenting the current ReAct-based architecture with more capable planning frameworks that support better long-horizon reasoning, error recovery, and reduced dependence on manual user input for parameter specification.

Section 6: Discussion

The case studies demonstrate that MetaPilot’s dual-mode architecture addresses two meaningfully different phases of a GSM study within a single interface. MetaInteract performs well for exploratory, ad hoc queries where the user’s intent evolves across turns and individual tool calls suffice. MetaPlan, by contrast, handles scenarios where the full analytical pipeline must be specified, reviewed, and executed in a principled sequence. The human-in-the-loop review step — in which the generated plan is inspected and edited before any execution begins — is a critical safeguard: it ensures that the final workflow reflects the researcher’s scientific intent rather than the model’s unchecked inference, and is what separates MetaPlan from prior agentic tools that offer no such checkpoint.

The PKB pipeline results underscore the value of grounding MetaPlan’s generation in retrieved published workflows rather than relying solely on the LLM’s parametric knowledge. General-purpose language models are known to produce plausible but methodologically unsound procedures when reasoning about specialised domains; anchoring plan generation in structured knowledge extracted from real GSM literature directly mitigates this failure mode. The publisher-aware section extractor — covering Springer, Wiley, Elsevier, Nature, and PMC layouts — proved essential for achieving consistent methods-section recall across the corpus. Naive extraction strategies fail systematically on significant subsets of a multi-publisher corpus; the custom extractor ensures that the downstream step-extraction stage operates on complete, noise-free methodological text.

The seven-category step taxonomy and the four-tier objective hierarchy together give the knowledge base a degree of semantic organisation that simple flat retrieval would lack. Retrieving methodologically coherent sub-workflows — rather than isolated, decontextualised steps — produces more actionable plans and is a key reason MetaPlan’s generated workflows align well with published GSM practice.

MetaPilot occupies a position in the GSM software landscape that no existing tool fills: end-to-end, natural-language-driven workflow execution grounded in literature. OptFlux, CNApy, Fluxer, and COBRAXy each address a subset of the modelling pipeline in isolation, offer no conversational interface, and have no capacity to generate multi-step plans from a user’s stated research objective. The pluggable LLM provider design further distinguishes MetaPilot from these tools by ensuring it is not bound to any single commercial API, permitting deployment with locally hosted models in settings where data confidentiality is a concern.

Section 7: Limitations

While the system demonstrates meaningful automation of GSM workflows, several limitations remain in the current implementation.

Retrieval Boundary Leakage in MetaPlan

The retrieval system occasionally surfaces steps that extend beyond the scope of the user’s query. For instance, a query requesting a context-specific model construction workflow may return results from papers where context-specific model generation is a precursor to downstream analyses such as flux balance analysis or flux variability analysis. Because the planner synthesises a workflow from these retrieved steps, the generated plan may include those subsequent analytical steps even when the user only asked for the model construction itself. This occurs because the knowledge base encodes complete experimental pipelines, and the semantic similarity between adjacent workflow stages can cause the retriever to pull in steps that are contextually related but not directly requested. The result is a plan that overshoots the user’s intent, requiring the user to manually identify and discard the extraneous steps.

Manual Parameter Specification

Despite automating the sequencing and orchestration of GSM operations, a non-trivial degree of user involvement is still required at execution time. Tool calls generated by the agent frequently contain placeholder or incomplete parameter values — such as reaction identifiers, bound values, gene names, or file paths — that the user must supply before the step can be executed. This means the system reduces the burden of knowing *what* to do, but not entirely the burden of knowing *how* to configure each operation, which may still pose a challenge for users unfamiliar with the underlying COBRApy API or the specific model they are working with.

Absence of Multi-Model Session Management

The current implementation supports only a single active model within a session at any given time. Workflows that require maintaining two models simultaneously — for example, constructing a generic and a context-specific model, performing independent analyses on each, and then comparing the resulting flux distributions — cannot be executed within a single session. A user wishing to perform such a comparative study would need to open two separate tool instances, conduct each sub-workflow independently, and reconcile the results manually. This is a significant architectural constraint for common GSM use cases such as condition comparison or strain benchmarking, where operating on multiple models in a coordinated fashion is central to the scientific question.

Section 8: Data and Code Availability

The MetaPilot web application is openly available at https://github.com/NiravBhattLab/Agenti_AI_for_COBRA/tree/MetaPilot. The Procedural Knowledge Base (PKB) is publicly released as a dataset on Hugging Face at https://huggingface.co/datasets/sistasathvik/gsm_procedural_knowledge_base. The PKB construction pipeline together with all analysis scripts used to produce the results in this thesis, are available at <https://github.com/saathviksista/MetaPilot-analysis-data-generation>.

List of Abbreviations

Abbreviation	Definition
API	Application Programming Interface
BAAI	Beijing Academy of Artificial Intelligence
BGE	BAAI General Embedding
BIA	Bioinformatics Agent
BiGG	Biochemically, Genetically and Genomically structured (database)
COBRA	Constraint-Based Reconstruction and Analysis
COBRApy	Constraint-Based Reconstruction and Analysis in Python
CORDA	Cost Optimization Reaction Dependency Assessment
CPLEX	IBM ILOG CPLEX Optimizer
EFM	Elementary Flux Mode
FBA	Flux Balance Analysis
FASTA	Nucleotide/amino-acid sequence file format (FAST-All)
FVA	Flux Variability Analysis
GEM	Genome-Scale Metabolic Model
GGUF	GPT-Generated Unified Format (llama.cpp model serialisation format)
GLPK	GNU Linear Programming Kit
GSM	Genome-Scale Metabolic
HR@3	Hit Rate at rank 3 (retrieval metric)
LLM	Large Language Model
MCS	Minimal Cut Set
MFA	Metabolic Flux Analysis
MOMA	Minimization Of Metabolic Adjustment
MRR	Mean Reciprocal Rank
P@3	Precision at rank 3 (retrieval metric)
PATRIC	Pathosystems Resource Integration Center
PKB	Procedural Knowledge Base
PMC	PubMed Central
RAG	Retrieval-Augmented Generation
ReAct	Reasoning and Acting (LLM agent framework)
RLC	Research Literature Corpus
ROOM	Regulatory On/Off Minimization
SBML	Systems Biology Markup Language
scRNA-seq	Single-cell RNA Sequencing
TDM	Text and Data Mining

Bibliography

- [1] Keren Yizhak, Barbara Chaneton, Eyal Gottlieb, and Eytan Ruppin. Modeling cancer metabolism on a genome scale. *Molecular Systems Biology*, 11(6):817, 2015. doi: 10.15252/msb.20145307.
- [2] Anuja Sathe, Susan M Grimes, Billy T Lau, Jiamin Chen, Carlos Suarez, Robert J Huang, George Poultides, and Hanlee P Ji. Single-cell genomic characterization reveals the cellular reprogramming of the gastric tumor microenvironment. *Clinical Cancer Research*, 26(11):2640–2653, 2020. doi: 10.1158/1078-0432.CCR-19-3231.
- [3] Stefanía Magnúsdóttir, Almut Heinken, Laura Kutt, Dmitry A Ravcheev, Eugen Bauer, Alberto Noronha, Kacy Greenhalgh, Christian Jäger, Joanna Baginska, Paul Wilmes, et al. Generation of genome-scale metabolic reconstructions for 773 members of the human gut microbiota. *Nature Biotechnology*, 35(1): 81–89, 2017. doi: 10.1038/nbt.3703.
- [4] Almut Heinken, Johannes Hertel, Ganesh Acharya, Dmitry A Ravcheev, Malgorzata Nyga, Onyedika Emmanuel Okpala, Marcus Hogan, Stefanía Magnúsdóttir, Filippo Martinelli, Monika Bhattacharyya, et al. Genome-scale metabolic reconstruction of 7,302 human microorganisms for personalized medicine. *Nature Biotechnology*, 41(9):1320–1331, 2023. doi: 10.1038/s41587-022-01628-0.
- [5] Eduard J Kerkhoven, Kyle R Pomraning, Scott E Baker, and Jens Nielsen. Regulation of amino-acid metabolism controls flux to lipid accumulation in *Yarrowia lipolytica*. *npj Systems Biology and Applications*, 2(1):16005, 2016. doi: 10.1038/npjbsba.2016.5.
- [6] Archana Hari and Daniel Lobo. Fluxer: a web application to compute, analyze and visualize genome-scale metabolic flux networks. *Nucleic Acids Research*, 48(W1):W427–W435, 2020. doi: 10.1093/nar/gkaa409.
- [7] Isabel Rocha, Paulo Maia, Pedro Evangelista, Paulo Vilaça, Simão Soares, José P Pinto, Jens Nielsen, Kiran R Patil, Eugénio C Ferreira, and Miguel Rocha. OptFlux: an open-source software platform for *in silico* metabolic engineering. *BMC Systems Biology*, 4(1):45, 2010. doi: 10.1186/1752-0509-4-45.
- [8] Sven Thiele, Axel von Kamp, Pavlos Stephanos Bekiaris, Philipp Schneider, and Steffen Klamt. CNApy: a CellNetAnalyzer GUI in Python for analyzing and designing metabolic networks. *Bioinformatics*, 38(5):1467–1469, 2022. doi: 10.1093/bioinformatics/btab828.
- [9] Francesco Lapi, Luca Milazzo, Lihao Lin, Isabella Cecilia Rizzo, Bruno Giovanni Galuzzi, and Chiara Damiani. COBRAxy: constraint-based metabolic modelling in Galaxy. *Bioinformatics*, 42(2):btaf670, 2026. doi: 10.1093/bioinformatics/btaf670. Published online December 2025.
- [10] Hsuan-Han Chiu, Ashley Varghese, Kunming Shao, Yen-Chun Lu, Rahul Nahar, Hao Chen, Qing Deng, Xiaoping Bao, and Can Li. scChat: A large language model-powered co-pilot for contextualized single-cell RNA sequencing analysis. *bioRxiv*, 2024. doi: 10.1101/2024.10.01.616063.
- [11] Huazhen Huang et al. ProtChat: An AI multi-agent for automated protein analysis leveraging GPT-4 and protein language model. *Journal of Chemical Information and Modeling*, 65(1):62–70, 2025. doi: 10.1021/acs.jcim.4c01345. Published online December 2024.
- [12] Alireza Ghafarollahi and Markus J Buehler. ProtAgents: Protein discovery via large language model multi-agent collaborations combining physics and machine learning. *Digital Discovery*, 3:1389–1409, 2024. doi: 10.1039/D4DD00009A.
- [13] Andrés M Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. Augmenting large-language models with chemistry tools. *Nature Machine Intelligence*, 6:525–535, 2024. doi: 10.1038/s42256-024-00832-8.

- [14] Yihang Xiao, Jinyi Liu, Yan Zheng, Xiaohan Xie, Jianye Hao, Mingzhi Li, Ruitao Wang, Fei Ni, Yuxiao Li, Jintian Luo, Shaoqing Jiao, and Jiajie Peng. CellAgent: An LLM-driven multi-agent framework for automated single-cell data analysis. *arXiv preprint arXiv:2407.09811*, 2024. doi: 10.48550/arXiv.2407.09811.
- [15] Kexin Huang, Serena Zhang, Hanchen Wang, Yuanhao Qu, Yingzhou Lu, Yusuf Roohani, Ryan Li, Lin Qiu, Gavin Li, Junze Zhang, et al. Biomni: A general-purpose biomedical AI agent. *bioRxiv*, 2025. doi: 10.1101/2025.05.30.656746.
- [16] Juexiao Zhou, Bin Zhang, Guowei Li, Xiuying Chen, Haoyang Li, Xiaopeng Xu, Siyuan Chen, Wenjia He, Chencheng Xu, Liwei Liu, and Xin Gao. An AI agent for fully automated multi-omic analyses. *Advanced Science*, 11(44):2407094, 2024. doi: 10.1002/advs.202407094.
- [17] Qi Xin, Quyu Kong, Hongyi Ji, Yue Shen, Yuqi Liu, Yan Sun, Zhilin Zhang, Zhaorong Li, Xunlong Xia, Bing Deng, and Yinqi Bai. BioInformatics agent (BIA): Unleashing the power of large language models to reshape bioinformatics workflow. *bioRxiv*, 2024. doi: 10.1101/2024.05.22.595240.
- [18] Jeffrey D Orth, Ines Thiele, and Bernhard Ø Palsson. What is flux balance analysis? *Nature Biotechnology*, 28(3):245–248, 2010. doi: 10.1038/nbt.1614.
- [19] Ali Ebrahim, Joshua A Lerman, Bernhard O Palsson, and Daniel R Hyduke. COBRApy: CONstraints-Based reconstruction and analysis for Python. *BMC Systems Biology*, 7(1):74, 2013. doi: 10.1186/1752-0509-7-74.
- [20] Laurent Heirendt, Sylvain Arreckx, Thomas Pfau, Sebastián N Mendoza, Anne Richelle, Almut Heinken, Hulda S Haraldsdóttir, Jacek Wachowiak, Sarah M Keating, Vanja Vlasov, et al. Creation and analysis of biochemical constraint-based models using the COBRA Toolbox v.3.0. *Nature Protocols*, 14(3):639–702, 2019. doi: 10.1038/s41596-018-0098-2.
- [21] Zachary A King, Andreas Dräger, Ali Ebrahim, Nikolaus Sonnenschein, Nathan E Lewis, and Bernhard O Palsson. Escher: A web application for building, sharing, and embedding data-rich visualizations of biological pathways. *PLOS Computational Biology*, 11(8):e1004321, 2015. doi: 10.1371/journal.pcbi.1004321.
- [22] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- [23] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. URL <https://arxiv.org/abs/2005.11401>.
- [24] Yuanjie Lyu, Zihan Niu, Zheyong Xie, Chao Zhang, Tong Xu, Yang Wang, and Enhong Chen. Retrieve-plan-generation: An iterative planning and answering framework for knowledge-intensive LLM generation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4683–4702, 2024. doi: 10.48550/arXiv.2406.14979.
- [25] LlamaIndex. LlamaIndex: A data framework for LLM-based applications. <https://www.llamaindex.ai>, 2022. Software framework. Available at https://github.com/run-llama/llama_index.

- [26] Nikolaos Livathinos, Christoph Auer, Maksym Lysak, Ahmed Nassar, Michele Dolfi, Panagiotis Vagenas, Cesar Berrospi, Matteo Omenetti, Kasper Dinkla, Yusik Kim, Shubham Gupta, Rafael Teixeira de Lima, Valery Weber, Lucas Morin, Ingmar Meijer, Viktor Kuropiatnyk, and Peter W J Staar. Docling: An efficient open-source toolkit for AI-driven document conversion. *arXiv preprint arXiv:2501.17887*, 2025. doi: 10.48550/arXiv.2501.17887.
- [27] Georgi Gerganov et al. llama.cpp: LLM inference in C/C++. <https://github.com/ggerganov/llama.cpp>, 2023. Open-source software. Available at <https://github.com/ggerganov/llama.cpp>.
- [28] Ines Thiele, Thuy D Vo, Nathan D Price, and Bernhard Ø Palsson. Expanded metabolic reconstruction of *Helicobacter pylori* (iT341 GSM/GPR): an *In Silico* genome-scale characterization of single- and double-deletion mutants. *Journal of Bacteriology*, 187(16):5818–5830, 2005. doi: 10.1128/JB.187.16.5818-5830.2005.